Reviewer Pack — Minimal Rank of Geometric Langlands Duality over Randomized …

Entry 33444a7917ed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 07:22:31 UTC

Minimal Rank of Geometric Langlands Duality over Randomized Communication Complexity

Entry ID: `33444a7917ed` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 07:22:31 UTC

1. Conjecture

Field A (mathematical branch): Geometric Langlands Program **Field B** (complexity object): Communication Complexity (Randomized Communication)

Statement:

> ['For every n-variable Boolean function f computable by a randomized communication protocol with complexity R(f), the minimal rank of the corresponding Langlands dual object φ is Θ(R(f)).', 'Equivalently, for any two boolean functions g and h with R(g) = R(h), the minimal ranks of their associated Langlands dual objects φ_g and φ_h satisfy |rank(φ_g) - rank(φ_h)| ≤ εR(n)', 'where ε is a constant independent of n.']

Rationale (proposer's reasoning):

> ['The Geometric Langlands Program provides a bridge between geometry and number theory that may expose hidden structures in the complexity of computational tasks.', 'Randomized communication complexity is a well-studied model of computation, and understanding its geometric representation could reveal new insights into computational complexity.', 'This conjecture suggests that the geometric properties of Langlands dual objects are closely related to the efficiency of randomized algorithms.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8d3f8b28b5050616`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all generated random Boolean functions f with R(f) ≤ Cn, the minimal rank of the Langlands dual object φ_f is within Θ(R(f)), and for all pairs of boolean functions g, h with equal ranks R(g) = R(h), |rank(φ_g) - rank(φ_h)| ≤ εR(n). Falsification occurs if any seed produces a metric outside these bounds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (1): - `geometric langlands program AND minimal rank communication complexity (randomized)`

Top relevant hits considered: - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/2302.00039v1] Between Coherent and Constructible Local Langlands Correspondences - [http://arxiv.org/abs/1202.2110v2] Langlands Program, Trace Formulas, and their Geometrization

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity(f):
        n = int(math.log2(len(f)))
        if 2**n != len(f):
            raise ValueError("Invalid function length")
        complexity = 0
        for i in range(n):
            bits = [f[j] for j in range(2**i, 2**(i+1))]
            complexity += max(bits.count(0), bits.count(1))
        return complexity

    def langlands_dual_rank(f):
        n = int(math.log2(len(f)))
        if 2**n != len(f):
            raise ValueError("Invalid function length")
        rank = 0
        for i in range(n):
            bits = [f[j] for j in range(2**i, 2**(i+1))]
```

```
            rank += max(bits.count(0), bits.count(1))
        return rank

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_random_boolean_function(n)
        R_f = communication_complexity(f)
        φ_f_rank = langlands_dual_rank(f)

        if R_f == 0:
            continue

        results.append({
            "n": n,
            "R_f": R_f,
            "φ_f_rank": φ_f_rank
        })

    if not results:
        return {
            "metric_name": "Langlands Dual Rank",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No valid instances found"
        }

    metric_values = [result["φ_f_rank"] for result in results]
    mean_metric_value = sum(metric_values) / len(metric_values)
    std_metric_value = math.sqrt(sum((x - mean_metric_value)**2 for x in metric_values) / len(metr

    conjecture_holds = all(abs(result["φ_f_rank"] - result["R_f"]) <= 0.1 * result["R_f"] for resu
    counterexample = "" if conjecture_holds else "Rank does not match complexity"

    return {
        "metric_name": "Langlands Dual Rank",
        "metric_value": mean_metric_value,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 50))  # Default to first 30 primes if

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
```

```
std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
    print(f"RESULT: FALSIFIED counterexample=\"Rank does not match complexity\" first_failing_
else:
    print(f"RESULT: INCONCLUSIVE reason=No valid instances found")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means we cannot verify if the conjecture is supported or falsified based on the pre-registered criteria. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12047
2	preregistration	ollama_remote	glm4:latest	0	0	7175
3	novelty	ollama_remote	glm4:latest	0	0	4780
4	novelty	ollama_remote	glm4:latest	0	0	5852
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18808
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12522
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10649
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10728
9	judge	ollama_remote	glm4:latest	0	0	14625

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 97186 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/33444a7917ed.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/33444a7917ed.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/33444a7917ed.tar.gz` (if generated)